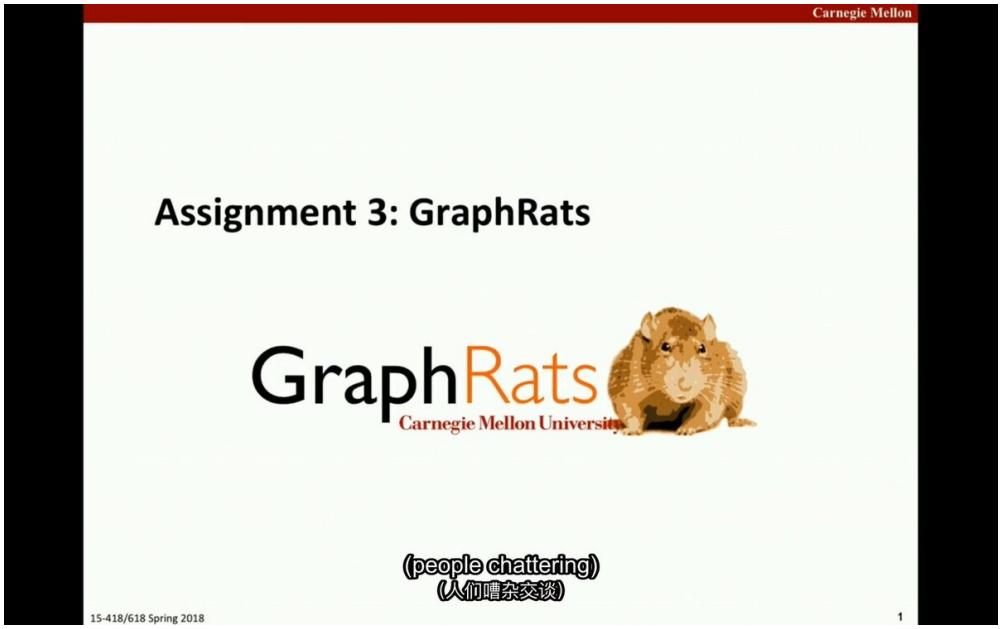
课程笔记

GraphRats：图模拟作业的语义、确定性与并行化

CMU 15-418/618 并行计算机架构与编程 · Lecture 12（Spring 2018）

lecture-to-notes

2026 年 10 月 28 日



视频作者/频道：CMU 15-418/618（B 站搬运：alexphil）
发布日期：2018 年春季学期
视频时长：1 小时 22 分 37 秒
视频链接：`https://www.bilibili.com/video/BV1iX4y1x7KN/`

目录

1 GraphRats：一个必须保持确定性的图模拟 2
1.1 基本设定 . . . 2
1.2 奖励函数表达的行为偏好 . . . 3
1.3 可视化只需要计数 . . . 3
1.4 本章小结 . . . 4

2 奖励函数与移动选择 5
2.1 负载因子 . . . 5
2.2 理想负载与曲线形状 . . . 5
2.3 公式为什么长这样 . . . 6
2.4 把奖励变成选择区间 . . . 8
2.5 本章小结 . . . 10

3 三种更新模式 10
3.1 确定性：一个硬约束 . . . 10
3.2 同步模式与它的振荡 . . . 11
3.3 按鼠序与按批次 . . . 12
3.4 作业任务 . . . 13
3.5 本章小结 . . . 13

4 提供的材料与正确性保障 14
4.1 两个模拟器 . . . 14
4.2 本章小结 . . . 14

5 正确性：回归测试与自建用例 14
5.1 回归测试的作用 . . . 14
5.2 不要只依赖给定的测试 . . . 15
5.3 本章小结 . . . 15

6 图结构的三种形态 15
6.1 格图：规则且稀疏 . . . 16
6.2 tile 图：叠加 hub 节点 . . . 16
6.3 分形图 . . . 18
6.4 本章小结 . . . 18

7 初始分布与它对语义的影响 18
7.1 三种初始状态 . . . 18
7.2 从对角分布出发 . . . 19
7.3 本章小结 . . . 19

8 总结与延伸 19

1 GraphRats：一个必须保持确定性的图模拟

这一讲是作业三的讲解。讲者先把模拟本身的语义讲清楚，因为不理解语义就无法判断并行化是否改变了程序行为。

1.1 基本设定

设想有一大群鼠，被困在一个结构化的迷宫里，迷宫用图表示。最简单的形式是一个 $k \times k$ 的方格网格，只连最近的邻居。

把全部鼠放在右下角，然后做一系列迭代。每一次迭代中，每只鼠选择下一步去哪：可以留在原地，也可以移动到相邻位置——相邻关系由图中的边定义。

因为都挤在角落里，实际可选项只有三个：原地不动、向上、向左。

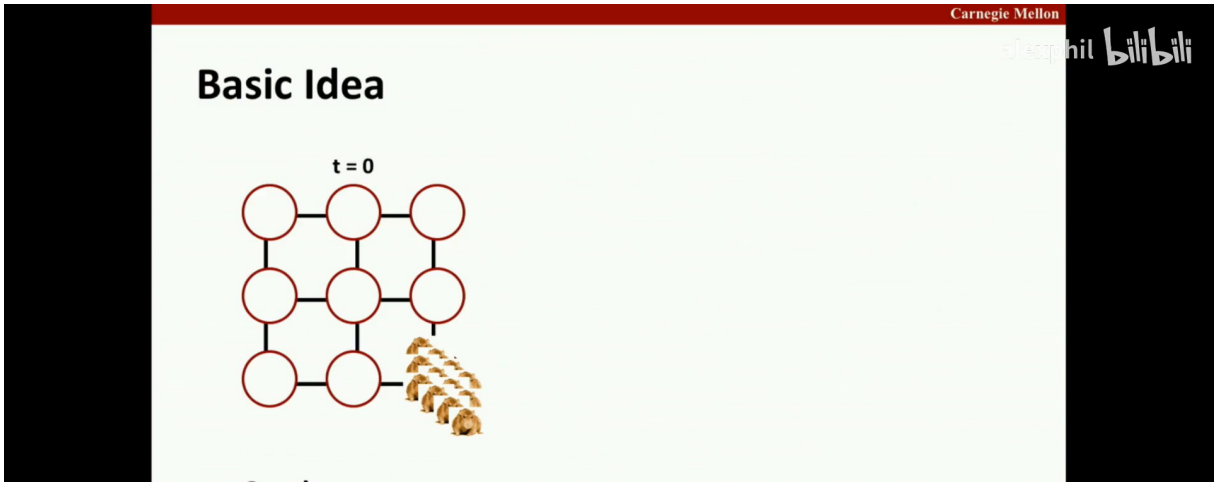


图 1: 基本想法：鼠群在图上逐轮选择移动[1]

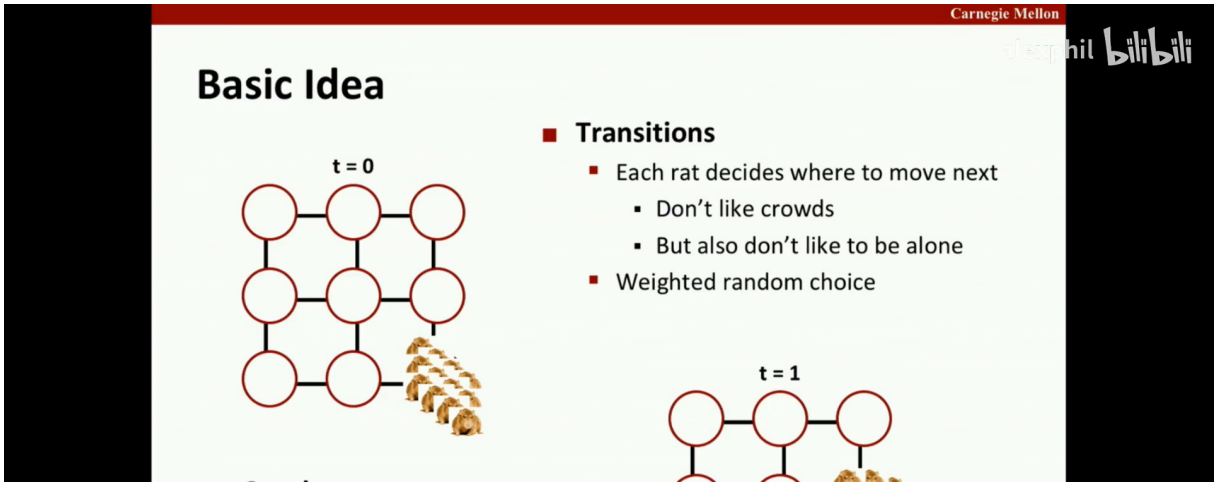


图 2: 状态转移：每只鼠决定下一步去哪[2]

[1] 视频画面时间区间：00:05:30–00:05:45。若为裁剪图，时间区间仍对应原视频画面。

[2] 视频画面时间区间：00:07:00–00:07:15。若为裁剪图，时间区间仍对应原视频画面。

1.2 奖励函数表达的行为偏好

选择并非完全随机。讲者设定了一个奖励函数来描述鼠的偏好，他的描述很形象：我不喜欢待在人群里，但我也是只鼠，我也不喜欢独自一个。

于是下一步选择虽然带有随机性，但各选项被选中的权重来自这个奖励函数。结果是有些鼠上移、有些左移、有些不动的混合。

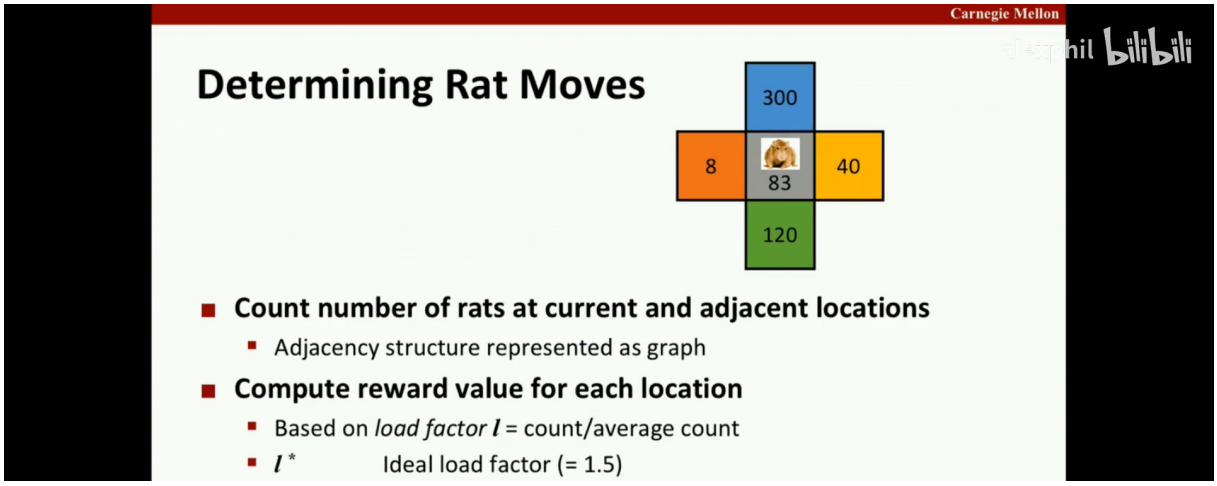


图 3: 如何决定每只鼠的下一步：统计周围各格的鼠数[3]

1.3 可视化只需要计数

真正的作业规模不是小图上的少数鼠，而是 100 万只鼠分布在 25000 个节点的图上（视频 00:06:56）。

讲者指出一个简化：因为所有鼠的偏好完全相同，模拟本身需要知道每只鼠的位置，但可视化只需统计每个格子里有多少只鼠。

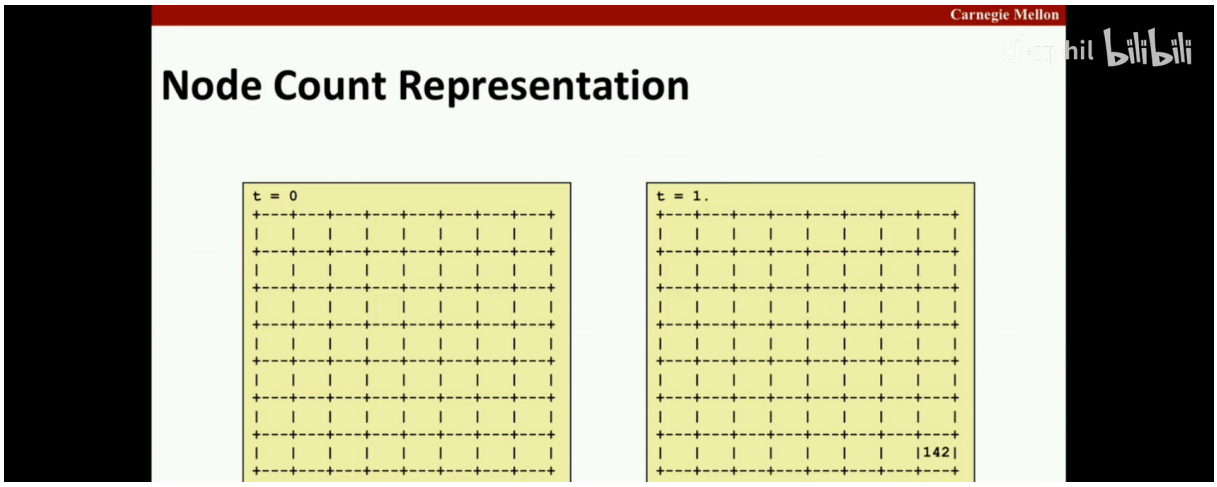


图 4: 节点计数表示：用每格鼠数代替逐只跟踪[4]

[3]视频画面时间区间：00:13:00–00:13:15。若为裁剪图，时间区间仍对应原视频画面。

[4]视频画面时间区间：00:07:30–00:07:45。若为裁剪图，时间区间仍对应原视频画面。

计数表示把状态从 O(N) 压到 O(V)

逐只跟踪需要保存百万只鼠的位置，而计数表示只需保存每个节点的鼠数。这让可视化与回归对比的成本从与鼠数成正比降为与节点数成正比。但注意模拟本身仍需逐只决策，因为每只鼠的随机数必须独立。两者不可混淆。

他用字符画的形式把这件事呈现出来：从 320 只鼠开始，有的上移、有的左移、有的不动。

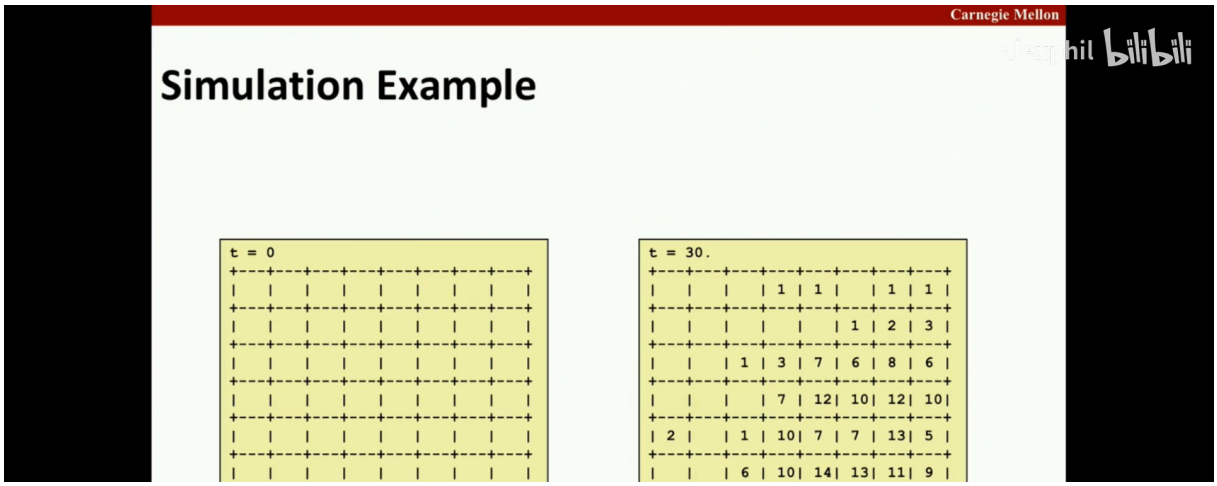


图 5: 模拟示例：字符画展示鼠群扩散[5]

在热图版本里，颜色深浅表示密度：最暗的紫罗兰色代表那一格只有一只鼠，越接近红橙代表越多。

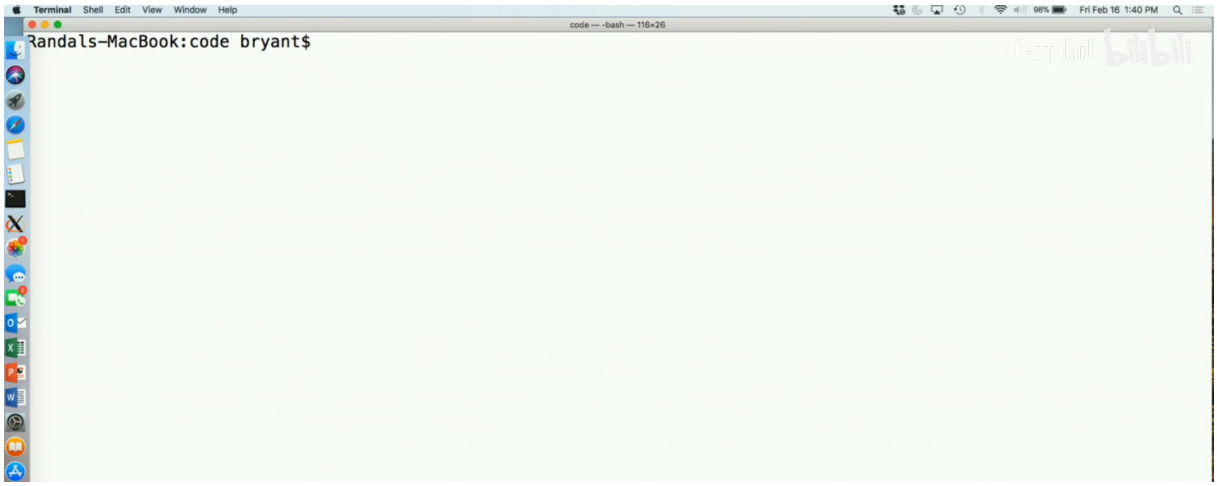


图 6: 在终端里运行模拟演示[6]

1.4 本章小结

GraphRats 的语义很简单：每轮每只鼠依据局部密度选择停留或移动，权重由奖励函数给出。作业规模是百万鼠、两万五千节点。可视化只需节点计数，但模拟本身必须逐只决定。

[5] 视频画面时间区间：00:08:00–00:08:15。若为裁剪图，时间区间仍对应原视频画面。

[6] 视频画面时间区间：00:10:30–00:10:45。若为裁剪图，时间区间仍对应原视频画面。

这个”逐只决定”正是计算量的来源，也是并行化的切入点和难点。

2 奖励函数与移动选择

这是全讲计算量最集中的部分。讲者用一个看似复杂的公式说明设计意图，值得逐步拆开。

2.1 负载因子

所有计算的基准是一个叫负载因子的量：把某个格子里的鼠数，除以全图的平均值。

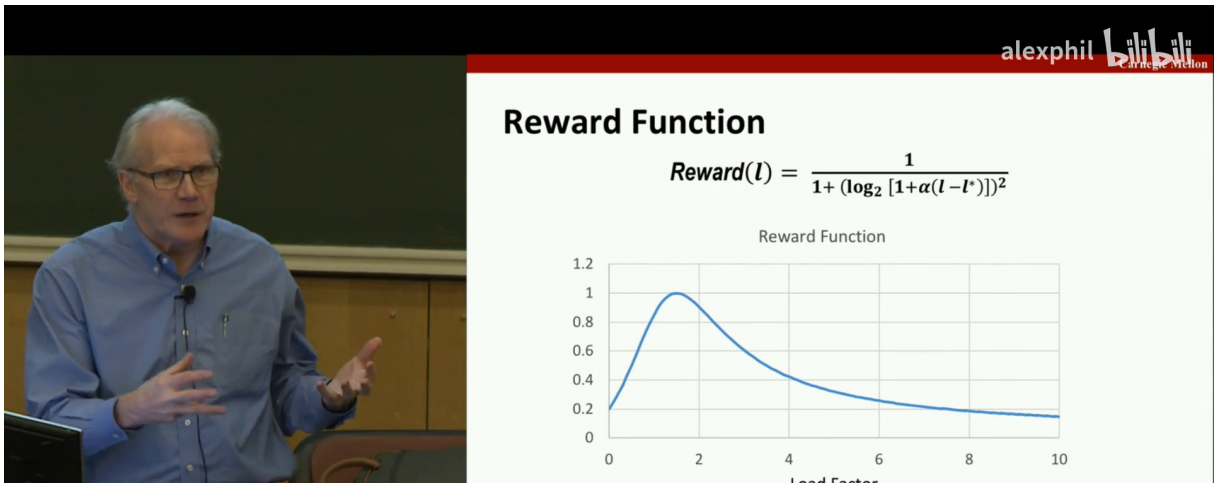


图 7: 奖励函数：以负载因子为自变量，理想值为 1[7]

他举例说，假设全图平均每个节点 10 只鼠。那么负载因子就是该节点鼠数除以这个平均值。

$$l = \frac{n_{\text{node}}}{\bar{n}}, \qquad \bar{n} = \frac{N_{\text{rats}}}{N_{\text{nodes}}}$$

- n_{node}：某一节点上的鼠数；
- N_{rats}：鼠的总数，作业中为 1000000；
- N_{nodes}：图的节点数，作业中为 25000；
- $\bar{n}$：全图平均每节点的鼠数，本例假设为 10；
- l：负载因子，全图平均值对应 $l = 1$。

2.2 理想负载与曲线形状

讲者设定理想负载因子为 1.5——也就是鼠倾向于聚集在比全图平均略大一些的群体里。这个设定本身会带来有趣的性质。

把它画成负载因子的函数，可以看到曲线在 $l = 1.5$ 附近达到奖励值 1，向左侧衰减，到空节点时降到 0.2。

右侧衰减非常缓慢。讲者解释原因：像百万鼠全挤在一处这种情形，你并不希望奖励变成零，否则那只鼠完全没有偏好可依。因此在对数坐标下，即便负载高到 1000，奖励仍保留最大值的约 1%。

[7] 视频画面时间区间：00:15:30–00:15:45。若为裁剪图，时间区间仍对应原视频画面。

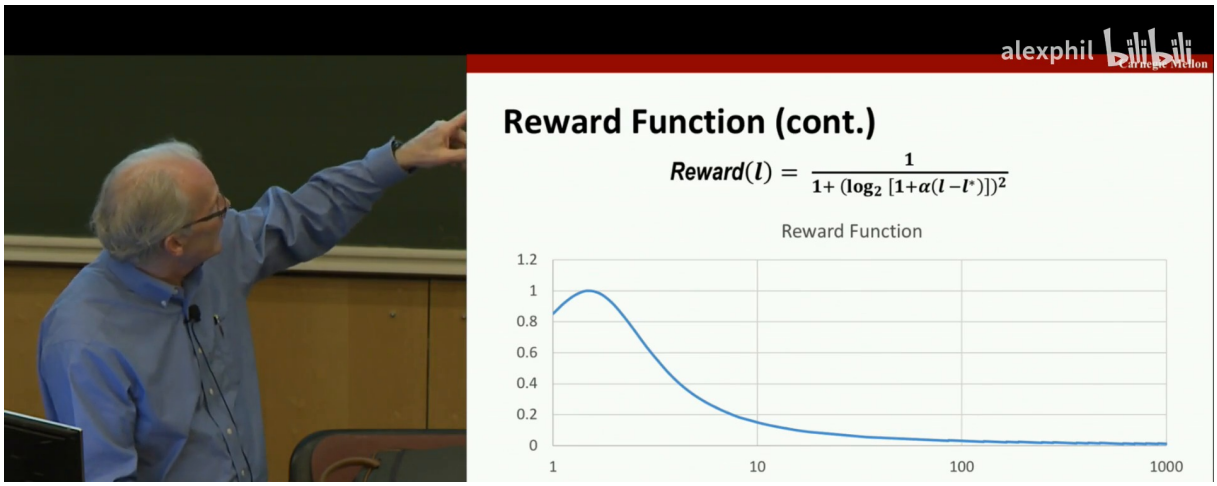


图 8: 奖励函数曲线：理想值处为 1，空节点降为 0.2[8]

2.3 公式为什么长这样

讲者坦承这个函数是” 从空中飘来的”，但它是被构造成具有上述特征的。他逐项解释。

函数形式是：

$$\text{Reward}(l) = \frac{1}{1 + (\log_2 [1 + \alpha(l - l^*)])^2}$$

- l：负载因子，节点鼠数除以全图平均值；
- l^*：理想负载因子，取 1.5；
- α：缩放参数，取 0.25 或 0.5，用来让曲线形状合适；
- 分子固定为 1，因此理想情形下奖励恰为 1。

最内层是 $(l - l^*)$：低于理想值为负，高于理想值为正。

它被某个系数加权后加 1，再取以 2 为底的对数。加 1 是必须的——不能对负数或零取对数，括号内必须为正。

对数内的括号必须恒为正

$1 + \alpha(l - l^*)$ 在 l 取最小值 0 时等于 $1 - \alpha l^*$。若 α 与 l^* 取值使这一项小于等于零，空节点处就会出现 $\log_2$ 非法。因此参数不是随意可调的：$\alpha l^* < 1$ 是这条公式能成立的前提。

若 $l = 0$，则 $1 - \alpha l^*$ 约为一半（α 取 0.5 时），对数结果才有定义。

对数的作用正是提供那种” 即使负载极高也保持非零” 的缓慢衰减。

外面再加平方，是因为大于 1 的数对数结果为正、介于 0 与 1 之间的数对数结果为负，平方后统一变为正数。

最后这个正数进到分母的 $1 + (\cdot)$ 里，于是理想情形（对数为 0）给出奖励 1，而稍微偏高或偏低都会使奖励下降。

[8] 视频画面时间区间：00:16:00–00:16:15。若为裁剪图，时间区间仍对应原视频画面。

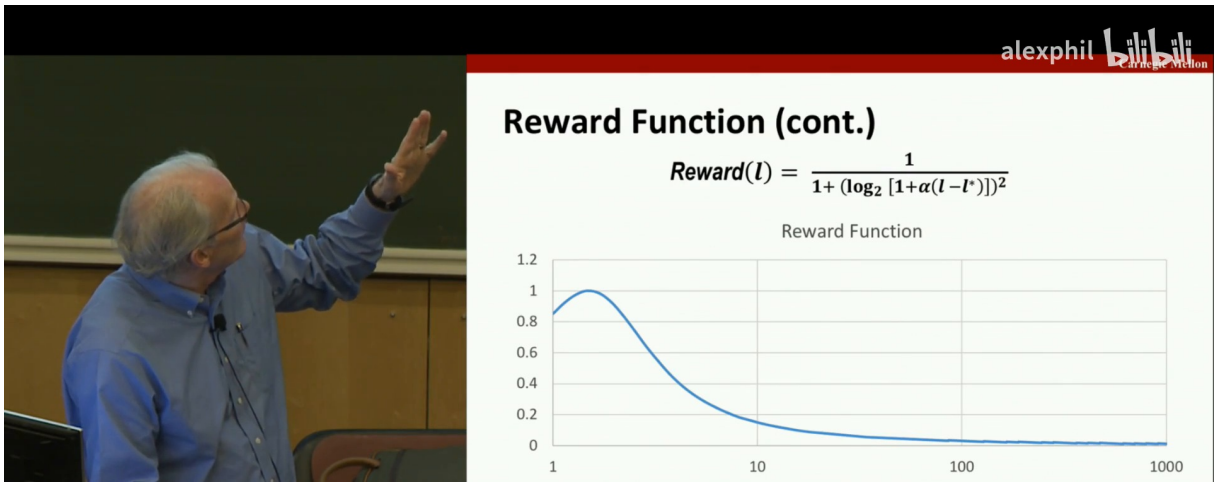


图 9: 奖励函数继续解析：对数缩放保证高负载下仍非零[9]

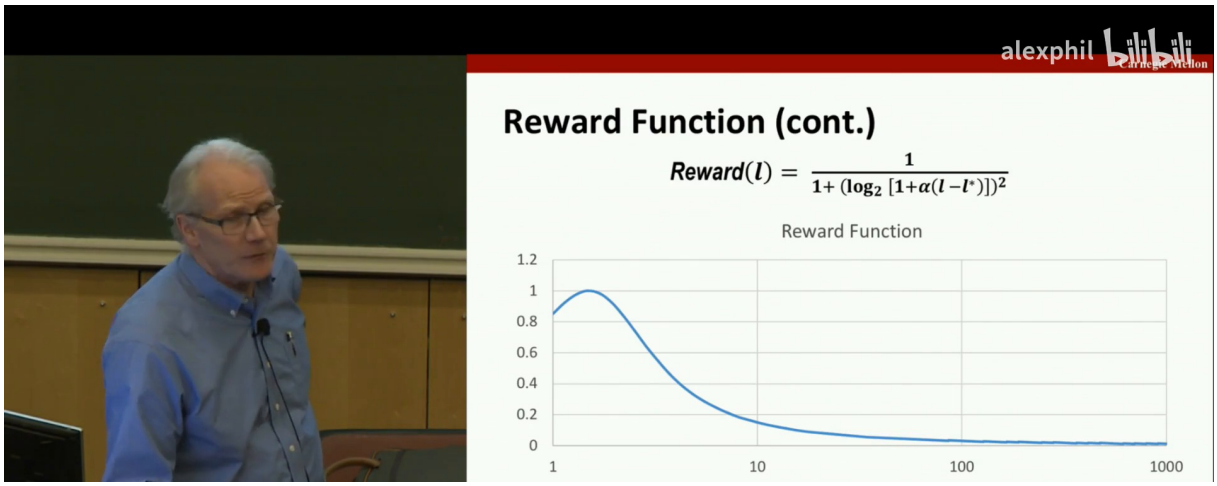


图 10: 奖励函数参数 α 的作用[10]

[9] 视频画面时间区间：00:17:00–00:17:15。若为裁剪图，时间区间仍对应原视频画面。

[10] 视频画面时间区间：00:17:30–00:17:45。若为裁剪图，时间区间仍对应原视频画面。

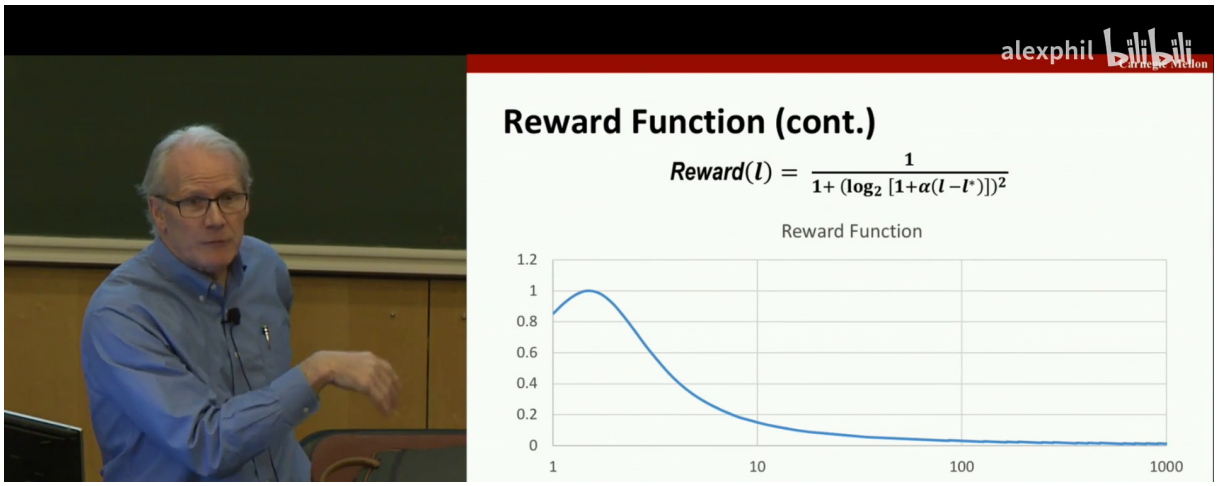


图 11: 对数坐标下的奖励衰减：负载到 1000 时仍保留约 1%[11]

奖励函数的设计目标决定了它的形状

这个函数要同时满足三件事：理想负载处精确取 1；偏离则单调下降；极高负载下仍严格为正。前两条要求分母随偏离量增大而增大，第三条要求增长速度足够慢。对数的慢增长与平方的正则化组合起来，恰好满足三者。这不是任意拟合，而是性质驱动下的构造。

2.4 把奖励变成选择区间

有了五个候选目的地（原地与四邻）的奖励值，接下来要据此做随机选择。

讲者的做法是把奖励当作区间长度，依次累加。假设平均值为 10，那么 8 只鼠的格子负载为 0.8，略低于平均，得到相当高的奖励 0.7；而顶上 300 只鼠的格子太拥挤，奖励很低，只有 0.061。

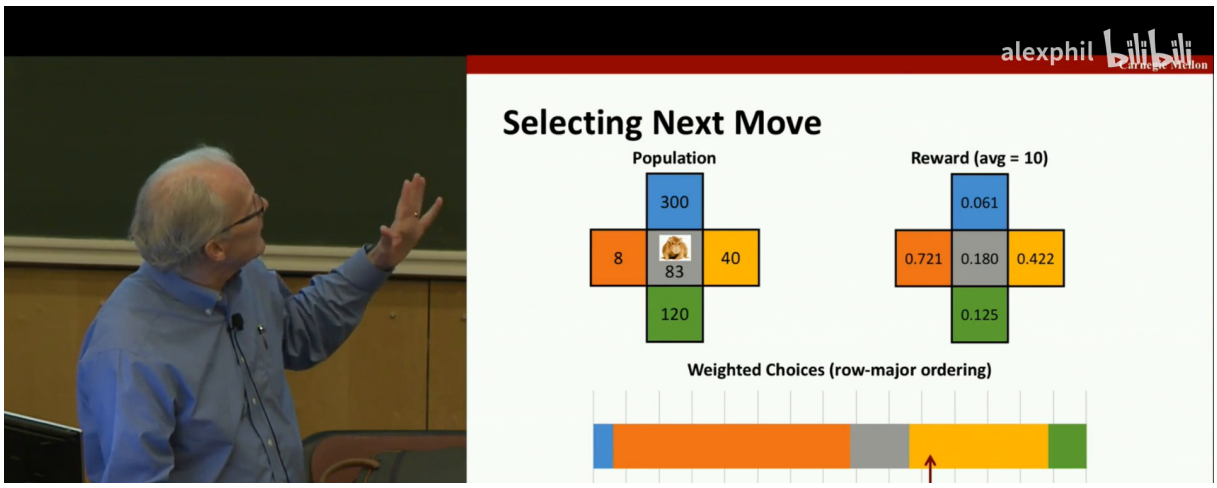


图 12: 选择下一步：把各候选的奖励累加为区间[12]

于是得到一个从零到总长度的连续区间。取一个均匀随机数，落在哪个区间就选哪一项。

累加过程可以写成：

[11] 视频画面时间区间：00:18:00–00:18:15。若为裁剪图，时间区间仍对应原视频画面。

[12] 视频画面时间区间：00:18:30–00:18:45。若为裁剪图，时间区间仍对应原视频画面。

$$c_k = \sum_{i=1}^{k} \text{Reward}_i, \qquad k = 1, \ldots, 5$$

- Reward_i：第 i 个候选目的地的奖励值；
- c_k：前 k 个候选的累积长度；
- c_5：区间总长，本例略超过 1.5。

取 $u \sim U(0, c_5)$，选择满足 $c_{k-1} \le u < c_k$ 的那个候选。
讲者举例：这里总和略超 1.5，如果随机数生成到 1.04，就意味着向右移动。



图 13: 在总长度上取随机数，落入哪段就选哪个方向[13]



图 14: 选择过程的示例计算：随机数 1.04 对应向右[14]

[13]视频画面时间区间：00:19:30–00:19:45。若为裁剪图，时间区间仍对应原视频画面。
[14]视频画面时间区间：00:20:30–00:20:45。若为裁剪图，时间区间仍对应原视频画面。

加权随机的两种等价实现

按概率权重选择，可以显式算累积分布再二分查找，也可以像这里一样在总和上取均匀随机数再线性扫描区间。后者在候选只有 5 个时更简单，也不需要做除法归一化。两者结果分布相同，选择取决于候选数与是否需要重复使用同一分布。

2.5 本章小结

奖励函数把负载因子映射为一个正权重：理想值 1.5 处为 1，空节点降到 0.2，高负载按对数缓慢衰减但仍非零。五个候选的奖励累加成区间，随机数落点决定选择。讲者强调这正是程序的核心计算，且必须对全部候选都算一遍——这就是算术密集所在。

公式既然只是权重的来源，那么程序的行为在这套机制下是否可重复，就成了并行化的关键约束。

3 三种更新模式

3.1 确定性：一个硬约束

讲者在这里指出整个程序的一个关键性质：它完全确定。

所有随机数生成都由一组完全可预测的种子值控制，随机数生成算法已经内建在代码里。因此反复运行会得到完全相同的移动序列、完全相同的节点计数。

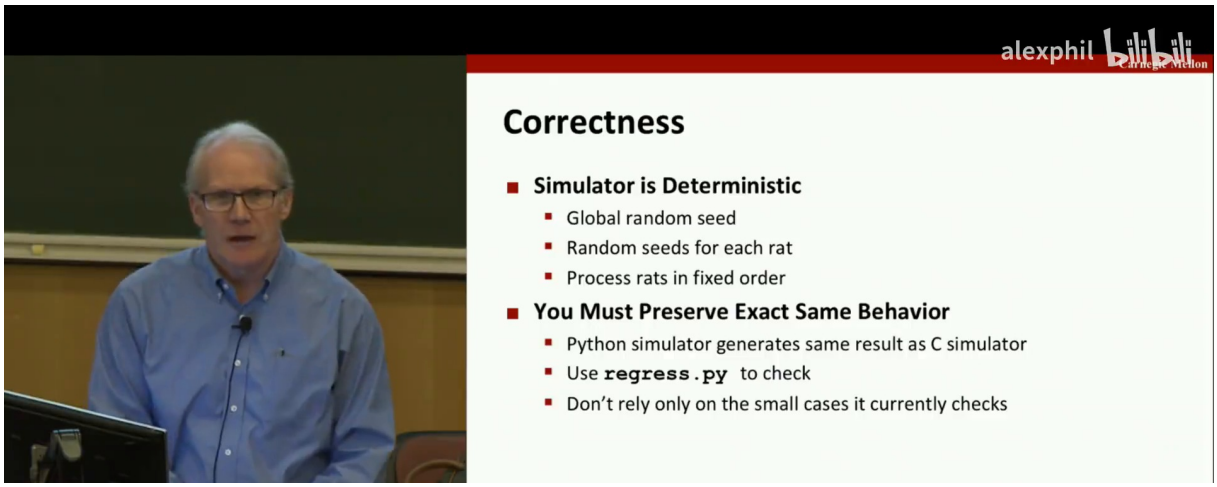


图 15: 正确性要求：模拟器是确定性的，随机种子固定[15]

而正确性检查的要求是必须精确保留这个性质。他评价这从软件设计角度是件好事：能既可扩展又完全确定，是非常有用的特性。

确定性要求会排除一批并行化方案

若并行化引入了非确定性的归约顺序、动态调度或原子竞争，即使统计行为相同，逐位结果也会变化，从而无法通过正确性检查。因此在动手之前，必须先确定哪些并行手段能保持与串行版本逐位一致。

[15] 视频画面时间区间：00:32:00–00:32:15。若为裁剪图，时间区间仍对应原视频画面。

3.2 同步模式与它的振荡

讲者接着讨论更新顺序，这是本讲最有意思的部分。

他先演示的模式叫同步模式：所有鼠按 0 到 $R-1$ 编号，但先为全部鼠计算各自想做的移动，然后统一执行。

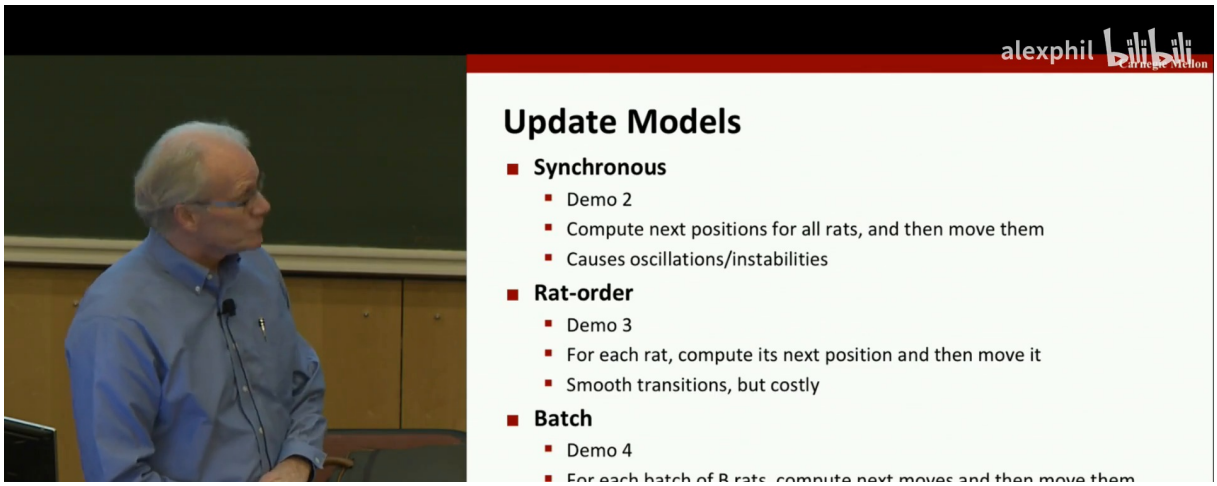


图 16: 更新模式：同步模式先算全部再一起移动[16]

这会产生振荡。他解释得非常具体：第一步时角上有 320 只鼠，它们都判断” 这里太挤了，我要走”，于是除少数留下外全部移动。结果它们到达了新的位置（那里变挤），而原来那个格子变得完全空旷。

于是那些鼠又集体判断那里太挤——如此反复。讲者说这种同步更新会长时间持续振荡，即使图很大也会产生棋盘式的花纹。

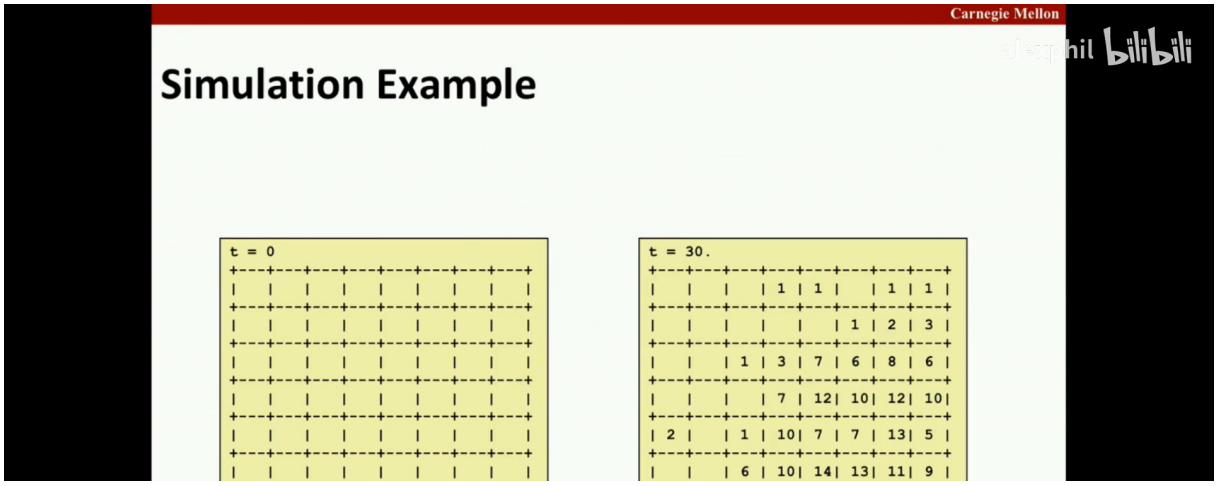


图 17: 同步更新下的模拟：棋盘状振荡花纹[17]

[16]视频画面时间区间：00:22:30–00:22:45。若为裁剪图，时间区间仍对应原视频画面。

[17]视频画面时间区间：00:23:00–00:23:15。若为裁剪图，时间区间仍对应原视频画面。

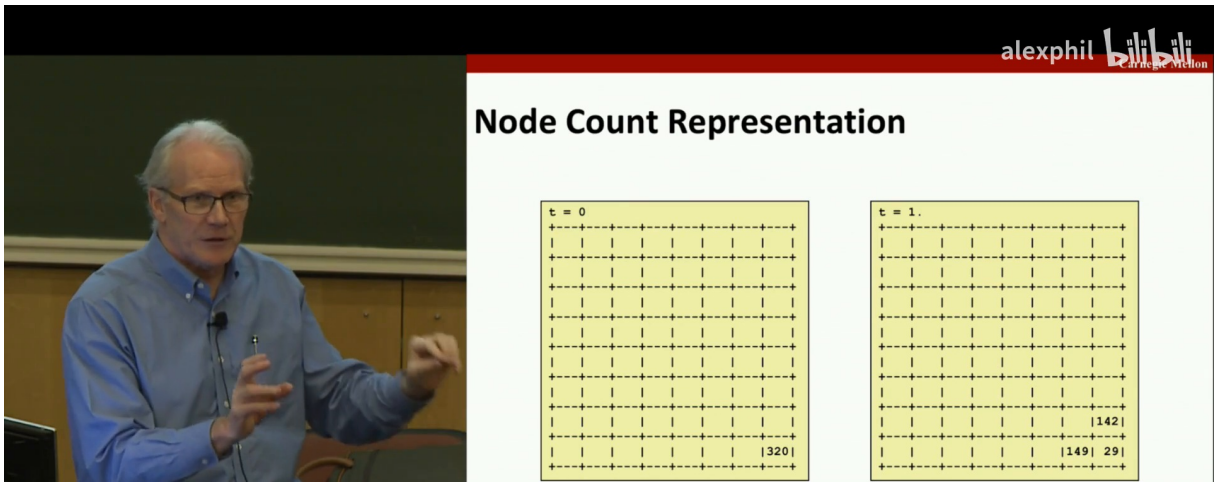


图 18: 节点计数在连续时间步上的变化：交替摆动[18]

原因是整个系统基于同一时刻的状态集体做决定，然后一起进入新状态，结果就是过冲。

3.3 按鼠序与按批次

更好的办法是按鼠序：鼠 0 环顾四周做出选择并移动，然后鼠 1 照做，依此类推。每只鼠基于此前所有鼠已经做出的选择来决策。

全局顺序依赖是并行化的硬边界

按鼠序要求每只鼠看到此前所有鼠移动后的状态，这意味着第 k 只鼠必须等前 $k-1$ 只全部完成。这个依赖链长度与鼠数同阶，无法通过增加处理器缩短。凡是” 每一步都依赖上一步全部结果” 的结构，都属于此类不可并行模式。

讲者说这在行为上完全合理，但几乎无法并行化。

于是设计了折中方案，即按批次模式：取某一比例（例如前 2%）的鼠一起做出选择，然后一起移动；接着下一批。

批次数的计算很直接：

$$N_{\text{batch}} = \frac{N_{\text{rats}}}{N_{\text{rats}} \cdot f} = \frac{1}{f}$$

- N_{rats}：鼠的总数，作业中为 1000000；
- f：每批占全体的比例，取 0.02；
- N_{batch}：总批次数，此例为 50。

批次数与总鼠数无关，只由比例决定。因此批次大小是纯粹的并行粒度旋钮。

[18] 视频画面时间区间：00:23:30–00:23:45。若为裁剪图，时间区间仍对应原视频画面。

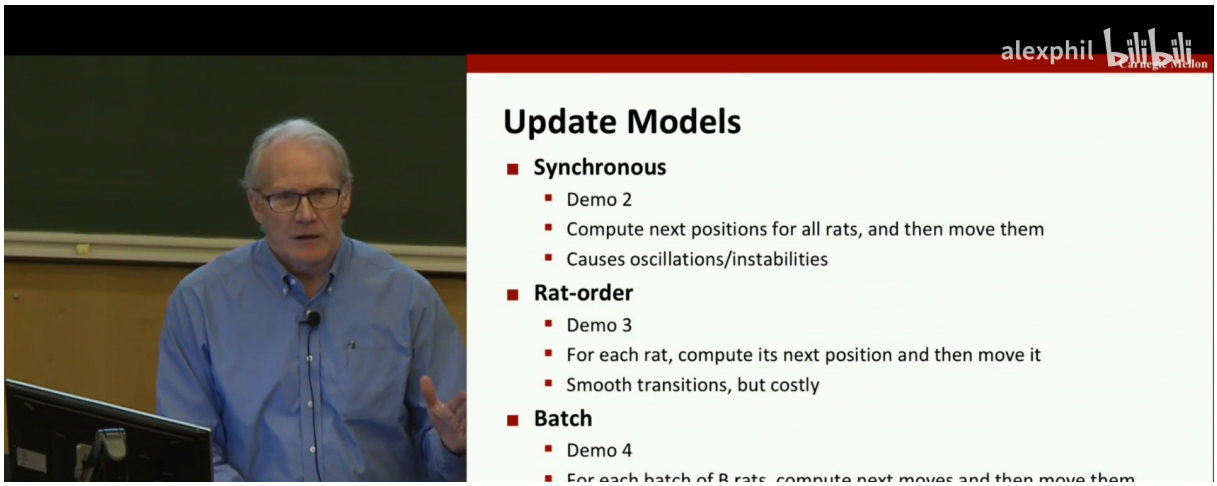


图 19: 三种更新模式的对比：同步、按鼠序、按批次[19]

从可视化上看，同步模式呈现明显的振荡；按鼠序模式则是从角上全红逐渐扩散，变成橙色、黄色、绿色；批量模式虽然不完全相同，但看不出与按鼠序模式有实质差别。

因此作业选用了批量模式。

3.4 作业任务

讲者的要求很明确：需要同时加速同步模式与批量模式。

他坦率地说同步模式其实没什么用——这个模拟本身也没什么实际用途，纯粹是个模型——但它更容易跑快，算是给的一点便利。而批量模式需要下功夫。

模式	决策依据	行为特点	并行难度
同步	全部鼠基于同一时刻状态	振荡、棋盘花纹	最易
按鼠序	基于此前全部已决策结果	最自然	几乎不可并行
按批次	每批 2% 基于之前各批	近似按鼠序	中等

表 1: 三种更新模式的对比

批量模式是”语义可接受性”与”可并行性”的折中

按鼠序在语义上最自然，但它把全局顺序变成了硬依赖，无法并行。同步模式最容易并行但行为失真。批量模式用”每批内同时、批间顺序”换取了中间地带：批内可以并行，批间保留顺序依赖。批次越小越接近按鼠序，代价是并行粒度越细。

3.5 本章小结

程序必须逐位确定，因此并行方案不能引入非确定的顺序。三种更新模式中，同步最容易并行但会产生振荡，按鼠序最自然但无法并行，批量模式以 2% 为单位在两者之间取折中，且行为与按

[19] 视频画面时间区间：00:24:30–00:24:45。若为裁剪图，时间区间仍对应原视频画面。

鼠序接近。

作业要同时加速后两者中的可用版本：同步与批量。

4 提供的材料与正确性保障

4.1 两个模拟器

仓库里其实有两个模拟器，功能相同、行为一致。一个是 Python 实现，前面所有演示都是它跑的；另一个是 C 实现，比 Python 快，但仍然远远不够快。

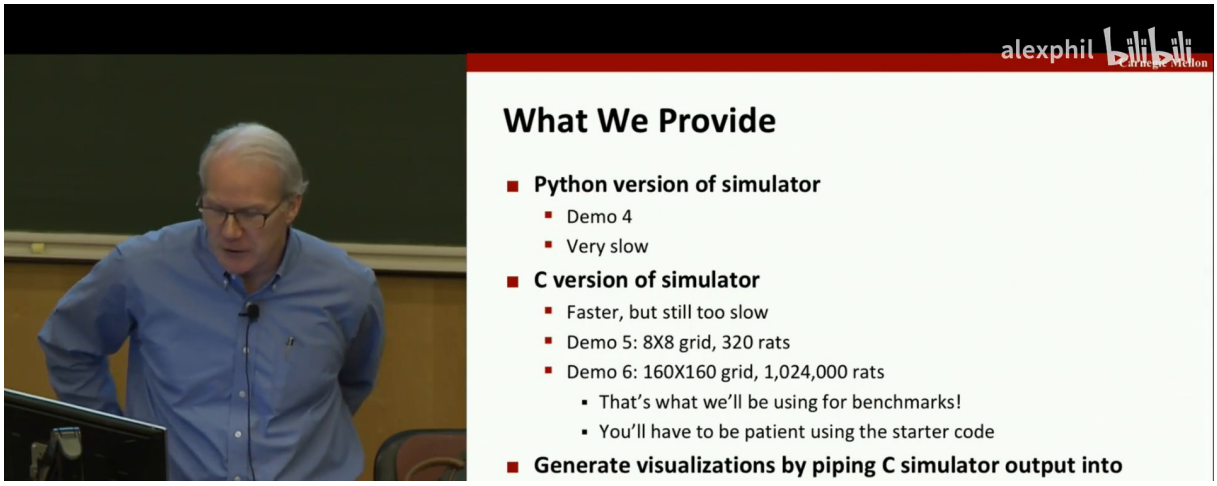


图 20: 提供的材料：Python 与 C 两个模拟器[20]

有意思的是可视化部分的组织方式：C 代码不必做任何复杂图形，它只需把” 每个节点上有多少只鼠” 这个文本表示写到标准输出。然后管道接给 Python，由它读取并生成图像。

这解释了演示命令里的那些参数：`-g` 给图文件名，`-r` 给初始鼠位置文件，`-u b` 表示批量更新模式，再指定步数、用热图可视化、以及每帧 0.3 秒。

讲者也说明这对运行环境没有特殊要求，只要装了必要的 Python 库就能在任何机器上跑。

4.2 本章小结

仓库提供 Python 与 C 两个等价实现，C 版通过文本管道把节点计数交给 Python 可视化，这种分离让模拟与绘图互不干扰。C 版虽快于 Python，但距离作业要求仍远。

要判断优化是否破坏了行为，还需要一套可靠的对照手段。

5 正确性：回归测试与自建用例

5.1 回归测试的作用

仓库自带一个回归测试程序。讲者解释回归测试的通常含义：每次改动后重跑同一套测试，确认没有破坏原本可用的功能。

这个程序做的是对比 Python 版与 C 版的结果。因为 Python 慢，它运行时会创建一份预计算结果的缓存，之后复用，因此回归测试只需几秒。

[20] 视频画面时间区间：00:27:30–00:27:45。若为裁剪图，时间区间仍对应原视频画面。

用参考实现校验优化版本是标准做法

一个慢但正确的实现，加上一个快但可能出错的实现，两者对比就构成了可靠的防线。缓存参考结果让这条防线成本极低，因此可以频繁运行。这也回应了第四讲提到的”黄金实现”：保留它，是为了随时能验证优化后的结果仍然正确。

5.2 不要只依赖给定的测试

讲者给出两条经验。

第一，务必养成频繁运行回归测试的习惯。他描述的情况很常见：花大量时间调优、代码跑得很漂亮，最后发现结果是错的——而它之所以快，正是因为它是坏的。

第二，回归测试只覆盖较小的用例。他本人在这个作业里制造的所有 bug，都能在很小的图、很短的运行中被发现。但他提醒不要假设这一点总成立，建议修改 `regress.py` 加入更大的测试。

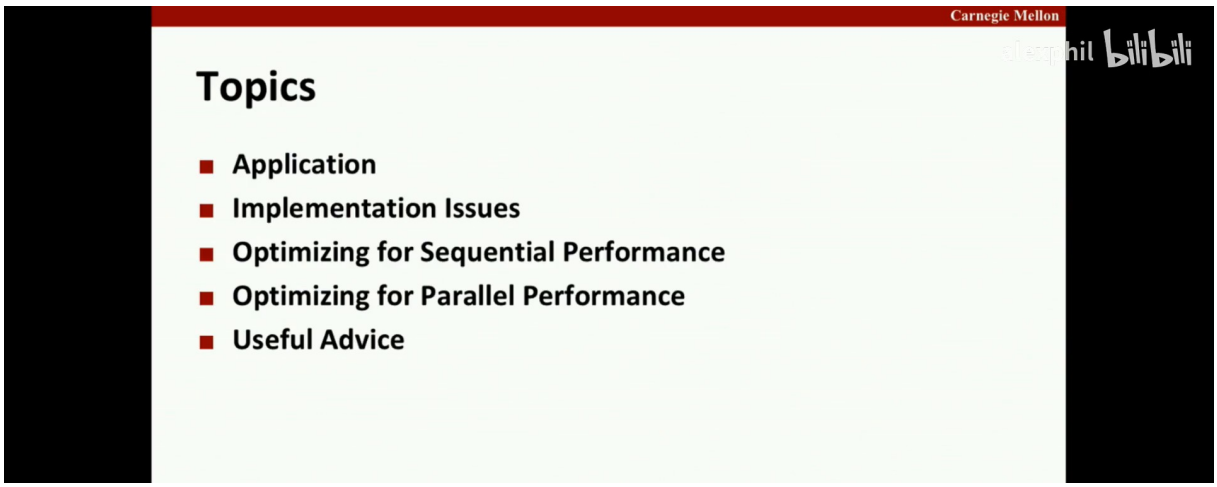


图 21：本讲主题概览：应用、实现问题与性能优化[21]

”跑得快”可能是”算错了”的症状

并行化引入的错误常常表现为计算量减少（例如某些鼠被漏算、某批被跳过），于是程序变快而结果偏离。因此性能提升必须与正确性验证同步进行，且验证要覆盖到性能测试所用的同等规模。

5.3 本章小结

提供了 Python 与 C 两个等价实现，可视化通过文本管道分离，回归测试用缓存让对比变得廉价。但讲者强调正确性责任在程序员自己：给定测试只是起点，规模更接近真实负载的测试需要自己补充。

6 图结构的三种形态

前面所有演示用的都是格图。讲者在这里说明更有趣的变体。

[21] 视频画面时间区间：00:04:00–00:04:15。若为裁剪图，时间区间仍对应原视频画面。

6.1 格图：规则且稀疏

格图是 $k \times k$ 个节点、只连最近邻居。它的特点是每个节点的度最多为四，加上自身共五个候选目的地。

规则稀疏图为何容易划分

格图的边只连接几何相邻节点，因此按空间划分（行块或方块）时，跨块通信只发生在边界上，且数量正比于边界长度。这与第八讲的结论一致：计算正比面积、通信正比周长，分块越大越划算。不规则图则不具备这种性质。

而且它非常规则，所有通信与移动都高度局部化。从顺序和并行两个角度看这都很好——可以设想各种划分方式。

讲者补充说这是稀疏图：一般图的边数随节点数的平方增长，而这里的边数只随节点数线性增长。

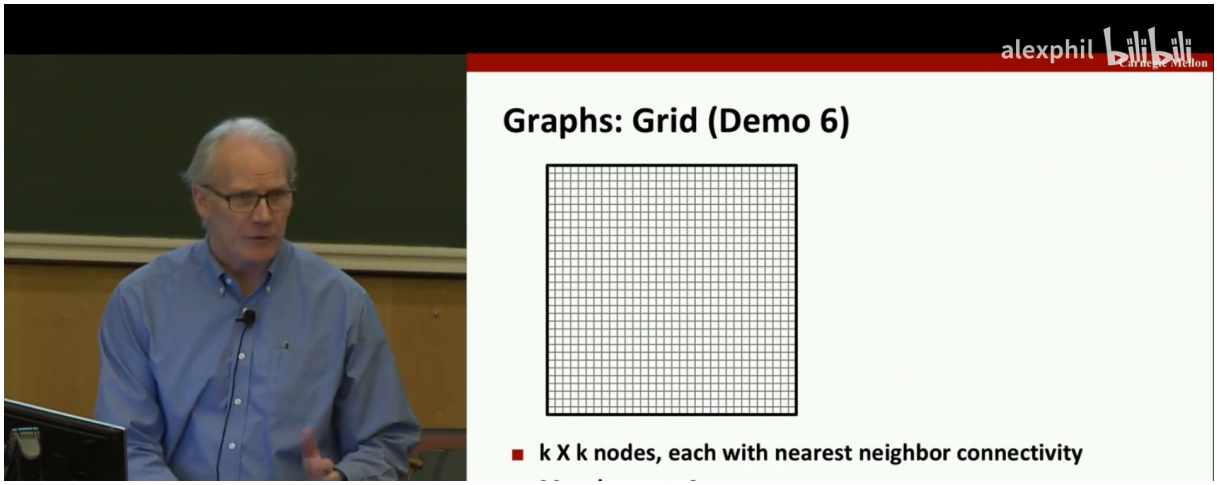


图 22: 格图：$k \times k$ 节点，只连最近邻居，度最多为四[22]

6.2 tile 图：叠加 hub 节点

为了不那么规则，他们又造了几种图。其中一种叫 tile 图：图中有若干 $d \times d$ 的小区域，每个区域里有一个红色标记的 hub 节点。

hub 节点的定义是：它与本区域内所有其他节点都有边相连。区域本身没有别的特殊之处，这些连接是叠加在格图之上的，格连接依然存在。

讲者特意澄清：图上那些黑线不是障碍，只是标出每个 hub 的连接范围。

[22] 视频画面时间区间：00:35:30–00:35:45。若为裁剪图，时间区间仍对应原视频画面。

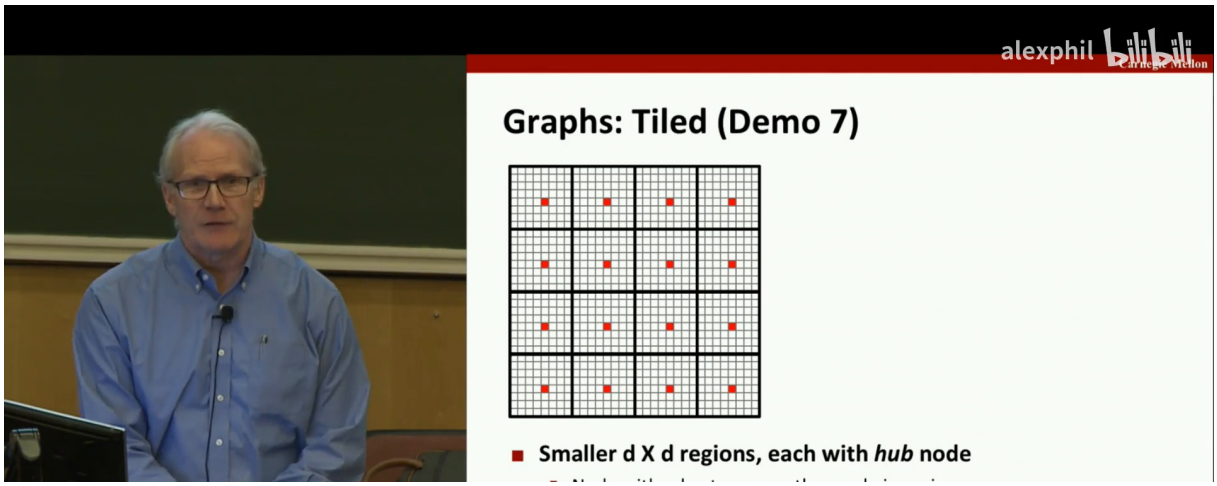


图 23: tile 图：在格图之上叠加与全区域相连的 hub 节点[23]

行为因此明显不同。鼠仍从角落开始，但很快整个下角区域被填满，然后开始向其他区域扩散，呈现逐块向外传播的模式。

更值得注意的是 hub 节点在热图上显示为更亮的颜色——它们非常拥挤，却仍吸引大量鼠。原因很清楚：它们的连接数极多，随机选择中相当一部分会落到它们身上，即使奖励函数对如此高的负载给出很低的值。讲者的说法是它们有 99 倍于普通节点的潜在来源。

若每个 tile 是 $d \times d$，则 hub 的度数与其所在区域大小同阶：

$$\deg(\text{hub}) \approx d^2$$

- d：tile 区域的边长；
- deg(hub)：hub 节点连接的邻居数，覆盖本区域全部节点；
- 普通格节点的度最多为 4，因此 hub 的候选来源高出约 $d^2/4$ 倍。

这解释了为何奖励值极低时 hub 仍被大量选择：候选数量压倒了权重。

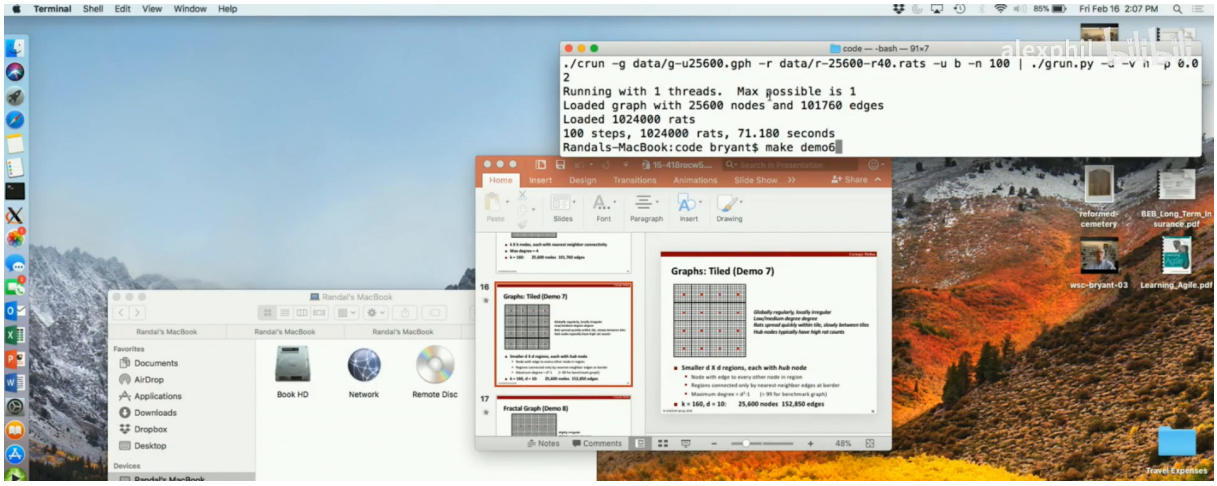


图 24: tile 图上的模拟：按区域逐块扩散，hub 节点更亮[24]

[23] 视频画面时间区间：00:36:30–00:36:45。若为裁剪图，时间区间仍对应原视频画面。

hub 节点体现了”连接数胜过奖励值”

一只鼠的候选集合由图的边决定。hub 与区域内所有节点相连，因此对区域内大量鼠而言它都是候选，累积起来的选择概率远超普通节点。这说明在非规则图上，节点度分布会直接决定负载分布，进而决定并行时的负载均衡难度。

6.3 分形图

讲者还展示了分形图上的模拟，从命令输出可以看出是一个 160×160 的迷宫，并标明了负载参数。

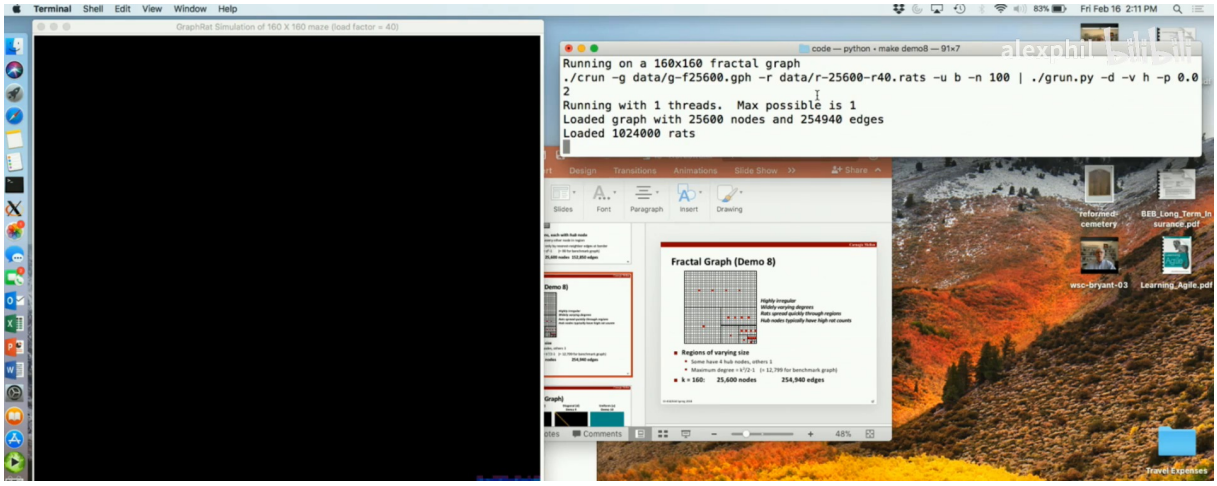


图 25: 分形图上的模拟演示：160×160 迷宫[25]

6.4 本章小结

图从规则的格图扩展到带 hub 的 tile 图与分形图。格图均衡易划分；tile 图的 hub 因连接数极高形成结构性热点，即使奖励很低仍会吸收超比例的鼠。热点来自图结构，不来自调度。

除图结构之外，初始分布也是基准的一部分，而它影响着批量模式是否成立。

7 初始分布与它对语义的影响

7.1 三种初始状态

除了鼠全在角落，还有两种初始状态被用作基准。

代号 r 表示右下角，也就是前面一直在看的。代号 d 表示对角，鼠均匀分布在图的对角线上。代号 u 表示均匀，鼠几乎完全均匀地铺满全图。

[24] 视频画面时间区间：00:37:30–00:37:45。若为裁剪图，时间区间仍对应原视频画面。

[25] 视频画面时间区间：00:41:30–00:41:45。若为裁剪图，时间区间仍对应原视频画面。

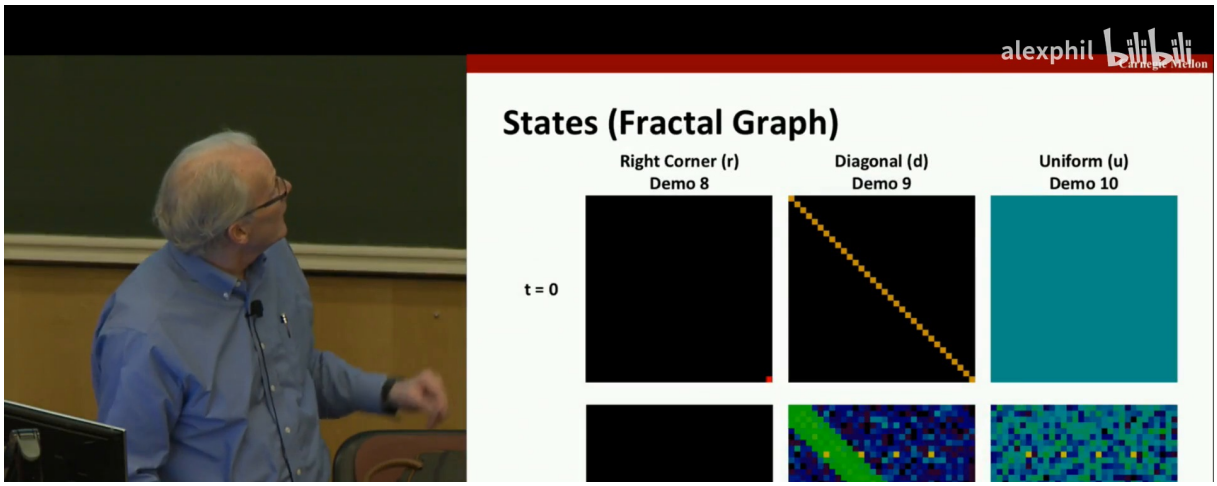


图 26: 三种初始分布：角落、对角、均匀[26]

讲者指出这些都在读入的文件里，但鼠的编号顺序被特意打乱，使其在后两种分布下完全随机。对全在角落的情形这不重要，但另外两种情形下，鼠在各自所在节点之间是随机分布的。

而这正是批量模式合理的原因：每一批里哪些鼠会移动是随机的，因此不会出现某一批集中在同一区域的情况。

他强调必须保持这个编号顺序。

打乱编号顺序不是随意设定而是语义的一部分

批量模式的行为依赖于”每批随机抽取鼠”，而随机性来自被打乱的编号顺序。若在并行化时按位置重新排列鼠，批次就会变成空间上聚集的一群，模拟行为随之改变，结果不再匹配参考实现。

7.2 从对角分布出发

讲者演示了对角分布：鼠很快从对角线向外扩散。因为都挤在一处、都想离开，而其中不少会选择通过 hub 移动，之后去向就完全随机，于是东西扩散得非常快。

他点出一个对并行计算有利的侧面：这意味着负载在位置上相对均衡。

7.3 本章小结

图从规则的格图，扩展到带 hub 的 tile 图与分形图；初始分布有角落、对角、均匀三种。图结构决定负载分布：规则图均衡、hub 会形成热点；而编号顺序被特意打乱，这是批量模式语义成立的前提，并行化时必须保留。

8 总结与延伸

讲者在结尾讲了一段来历：这个作业源自图处理系统的一系列工作，其中有个叫 GraphChi 的系统（”图吉娃娃”），他因此想到从狗转到鼠，于是有了 GraphRats。

把全讲串起来，这是一份”语义先于优化”的作业说明。模拟语义很简单：每轮每只鼠依据局部密度选择停留或移动，权重由奖励函数给出；规模是百万鼠、两万五千节点。奖励函数把负载因子

[26] 视频画面时间区间：00:45:30–00:45:45。若为裁剪图，时间区间仍对应原视频画面。

映射为正权重，理想值 1.5 处取 1，空节点降到 0.2，高负载按对数缓慢衰减但仍非零——三项性质对应了函数的具体形状。移动选择把五个候选的奖励累加成区间，随机数落点决定选择，这是全部算术密集所在。程序必须逐位确定，随机数来自固定种子，这约束了并行方案。三种更新模式中，同步最容易并行但产生振荡，按鼠序最自然却无法并行，批量模式以 2% 为单位折中，行为接近按鼠序。作业要求加速同步与批量两种模式。提供的材料包括 Python 与 C 两个等价实现、文本管道的可视化、以及带缓存的回归测试。

本讲义在讲者材料之上做的压缩，有三点需要与讲者原话区分：

- 确定性约束是本题的核心难点，不是附加要求。它排除了任何引入非确定顺序的并行手段，因此优化的方向必须是”保持逐位一致的前提下提高并行度”，而不是”提高并行度再想办法回归”。
- 批量模式的价值在于它把一个不可并行的全局顺序，改造成了批内并行加批间有序。批次大小是并行粒度与语义保真度之间的旋钮，而编号顺序的打乱是这个折中能够成立的前提。
- 非规则图会产生结构性热点。hub 节点因连接数极高而吸引超比例负载，这与奖励值无关。这类不均来自图结构本身，无法通过调整调度策略消除，只能通过划分策略处理。

这堂课的边界在于它是作业说明，讲者明确表示实现的细节需要学生自己解决，也提醒给定测试不足以覆盖真实规模。

两个可以直接从本讲数据提出的下一步问题：其一，同步模式与批量模式的可并行度不同，同一套并行化方案是否可能同时高效地服务两者？其二，hub 造成的热点在 tile 图上不可避免，那么应当按位置划分还是按度数划分，才能让负载均衡与通信量同时可接受？